Lab 2C: Precision & Recovery

Duration: 30-45 min | **Difficulty:** Intermediate

After: Context Feeding & @ Mentions slides

Continues from: Lab 2B (your Business Dashboard should have CLAUDE.md)

© 2026 AIA Copilot — Instructor-Led Training Material

Goal

Master context control with @ mentions to make Claude look exactly where you want — and learn to recover when things break.

Steps

1. Precision Refactoring with @ Mentions

Start with a broad prompt to see the difference:

```
Update the API to handle errors better.
```

Notice: Claude has to search your entire project, decide which files matter, and guess what "better" means. This burns tokens and produces generic results.

Now try with precision:

```
Look at @src/index.js — the routes are all in one file.
Refactor: extract metric routes into src/routes/metrics.js
and task routes into src/routes/tasks.js.
Keep the main file clean with just app setup, middleware, and route mounting.
```

Watch Claude: 1. Read the specific file you mentioned 2. Identify the route groups 3. Create two new files following your CLAUDE.md conventions 4. Update the main file to import and mount them 5. Test that nothing broke

> ***Why this matters:*** *Every token Claude reads costs money and uses limited context window space. @ mentions load exactly the files you need — nothing more. In a large codebase, the difference between* `@src/auth.ts` *and "look at the auth code" could be 50,000 tokens of unnecessary context.*

2. Verify Nothing Broke

Ask Claude to verify:

```
Test all the API endpoints — metrics, tasks, dashboard, and stats.
Show me the results.
```

Expected output: All endpoints return valid data. The refactor was invisible to the API consumer.

3. Intentional Break & Recovery

This is how developers actually use Claude Code — things break, and you fix them.

Ask Claude to simulate a problem:

```
Delete the file src/routes/metrics.js
```

Now recover:

```
The metrics routes file was deleted. Restore it based on the original
API spec and our CLAUDE.md conventions. Make sure all metric endpoints work.
```

Watch Claude: 1. Reads the remaining code to understand what's missing 2. Checks CLAUDE.md for conventions 3. Rebuilds the file following your rules 4. Tests that it works

What Just Happened?

Recovery prompting is a core skill. Claude's diff awareness means it can reconstruct what's missing by analyzing what's still there.

- Claude read `src/routes/tasks.js` to understand the pattern
- It checked `src/index.js` to see what routes were being imported
- It regenerated the metrics routes following the same conventions
- It verified everything still works

In the real world, you'll use this pattern when: - A merge conflict corrupts a file - You accidentally delete something - A dependency upgrade breaks existing code - You need to reverse-engineer a missing component

> ***The pattern:*** *Describe what's wrong + what the result should look like. Claude handles the detective work.*

---

4. Add a Feature Using Multiple File References

```
Look at @src/routes/tasks.js and @public/index.html — add the ability
to create a new task from the frontend UI. Add a simple form at the top
of the tasks section with fields for title, priority (dropdown), and
assigned_to. When submitted, POST to the API and refresh the task list.
```

5. Test the Form

Open `http://localhost:3100` in your browser. You should see a form above the task list. Create a task and verify it appears.

6. Commit

```
Commit these changes as two separate commits:
one for the refactor, one for the new feature
```

Watch Claude organize changes into logical groups and write descriptive conventional commit messages for each.

Optional: Go Deeper

URL @ mentions — bring in external docs:

```
@https://expressjs.com/en/guide/error-handling.html Update our error
handling middleware to match Express best practices from this guide
```

Claude will fetch the Express documentation, read the patterns, and update your code.

Multi-file investigation:

```
Look at @src/routes/metrics.js and @src/routes/tasks.js — are there any
patterns that could be extracted into a shared utility? If so, create
src/utils/response.js with shared helpers and refactor both route files to use it.
```

Stress test the recovery:

Delete TWO files at once:

```
Delete src/routes/metrics.js and src/routes/tasks.js
```

Then:

```
Both route files were deleted. Restore them both based on the API spec
in CLAUDE.md and the current state of src/index.js. Test everything.
```

The "explain it to me" pattern:

```
Walk me through @src/index.js line by line.
Explain what each section does and why it's structured this way.
```

This is great for learning codebases you didn't write.

□Lab 2C Checkpoint

- [ ] Routes split into separate files (src/routes/)
- [ ] Recovery from deleted file successful
- [ ] All API endpoints still work
- [ ] Frontend has a "New Task" form that creates tasks
- [ ] Two clean git commits (refactor + feature)

> ***Keep Claude running. Return to slides when your instructor is ready.***

© 2026 AIA Copilot — Instructor-Led Training Material

© 2026 AIA Copilot Training